

Relatório

Desenvolvimento de um Assistente de Pesquisa Inteligente para E-commerce

Miguel Grilo | Aluno 58387
Licenciatura em Inteligência Artificial e Ciência de Dados
Universidade de Évora

IMPACT Commerce
Orientadores: José Barbosa & Cristina Silva

Resumo Este relatório detalha a fundamentação matemática e algorítmica que sustenta o desenvolvimento de um Assistente de Pesquisa Inteligente para E-Commerce, desenhado para superar a disparidade semântica entre a linguagem humana e a indexação rígida de catálogos. A solução assenta num paradigma de **Pesquisa Híbrida**, combinando a precisão léxica do modelo probabilístico **BM25** com a compreensão latente de embeddings vetoriais, cuja proximidade geométrica é avaliada através da **Similaridade de Cosseno** e cuja recuperação eficiente é assegurada pelo algoritmo **HNSW**. A convergência destas métricas assimétricas é garantida pelo algoritmo **Reciprocal Rank Fusion (RRF)**. O núcleo cognitivo do sistema é suportado por um **Large Language Model** que, apoiado na arquitetura **Transformer** e nos seus mecanismos de **Self-Attention**, opera como um tradutor funcional: converte ambiguidades linguísticas numa dualidade rigorosa de vetores semânticos e restrições lógicas. O protótipo valida que a orquestração conjunta destes modelos matemáticos resolve as limitações da pesquisa convencional, garantindo uma recuperação de informação em tempo real que é simultaneamente fluida na interação e determinística na aplicação de regras de negócio.

Keywords: E-Commerce · NLP · LLM · Pesquisa Híbrida · BM25 · Similaridade de Cosseno · Reciprocal Rank Fusion · Azure AI Search.

Introdução

Estado da Arte do E-Commerce

Atualmente, o panorama do E-Commerce tem vindo a sentir a necessidade de evoluir graças aos avanços na área da Inteligência Artificial. A experiência dos utilizadores deixou de estar limitada a componentes estáticos, nomeadamente as vastas grelhas de produtos e os filtros de pesquisa, e passou a contar também com interfaces mais intuitivas, capazes de simular a experiência fluida e personalizada "em loja" do utilizador. No entanto, apesar dos avanços tecnológicos recentes, continuam a existir desafios no que toca à linguagem natural e ao seu entendimento por parte da IA.

Desafios da Disparidade Semântica & NLP

Estes desafios apresentam um obstáculo crítico para esta transição, nomeadamente a disparidade semântica entre a linguagem humana e os sistemas de indexação de produtos utilizados pelas lojas.

Os métodos de pesquisa/busca convencionais são baseados fundamentalmente na correspondência exata de palavras-chave, métodos estes que falham frequentemente aquando da necessidade de interpretar a intenção real por trás de consultas subjetivas/ambíguas, como questões simples que seriam entendidas facilmente por um humano, por exemplo, "quero algo casual para o verão", porém a pesquisa convencional não está preparada para lidar com estes casos corretamente. Assim, para lidar com estes casos, a aplicação de técnicas de Processamento de Linguagem Natural (NLP) entrou na equação do problema, tornando-se essencial para a interpretação das variações linguísticas e intenções implícitas do utilizador.

Objetivos do Protótipo & Breve Contexto da Arquitetura da Solução

O protótipo do Assistente Inteligente visa então colmatar esta lacuna através de uma interface capaz de reformular consultas complexas em tempo real e gerir o contexto em diálogos multi-turno na linguagem natural do utilizador.

Para tal, temos assente uma arquitetura modular organizada em camadas (Api, Application e Custom) e a solução utiliza o Azure AI Search para implementar uma estratégia de pesquisa híbrida que combina a robustez da pesquisa textual com a profundidade semântica de embeddings vetoriais, orquestrados por um LLM, garantindo que o motor de busca opere com máxima relevância técnica e semântica, proporcionando uma experiência personalizada e eficiente ao utilizador.

Gestão & Engenharia de Dados

Origem & Estrutura do Dataset

De modo a conferir realismo e eficácia ao projeto, a aquisição de um conjunto de dados que refletisse a complexidade de um catálogo de moda foi fundamental para realizar testes adequados. Embora se trate de um catálogo simulado, este replica fielmente a estrutura esperada para o catálogo de um cliente real. O conjunto de dados em questão contém, para cada produto e variante do mesmo, uma árvore rica de metadados, incluindo categoria, marca, entre outros.

A escolha de um catálogo em Português de Portugal permitiu validar a sensibilidade do sistema às nuances linguísticas locais, garantindo que o assistente compreenda termos específicos do mercado nacional.

Normalização, Padrões & Vocabulário

Este conjunto de dados em estado bruto exigiu uma atenção minuciosa no que toca à sua transformação para o índice de pesquisa. Foram delineados critérios de normalização para os atributos críticos, como o mapeamento de tamanhos — convertendo termos de linguagem natural (ex: "pequeno" ou "grande") para os valores padronizados do sistema (S ou L) — e a uniformização de categorias. Além disso, foram injetadas `CatalogRules` diretamente no contexto do LLM, restringindo o vocabulário do modelo para determinados atributos com valores existentes no inventário real, o que mitiga drasticamente a ocorrência de alucinações durante a filtragem.

Engenharia de Dados

A transformação técnica foi realizada através de um `ProductSearchDocumentMapper`, um componente responsável por decompor os objetos JSON em campos otimizados para o *Azure AI Search*. Esta camada assegura a integridade da pesquisa híbrida ao separar o conteúdo para pesquisa léxica daquele destinado à geração de vetores. Além disso, permite que metadados estruturados sejam indexados de forma a suportar filtros OData com elevada performance.

Fundamentação Matemática & Metodológica

Paradigma da Pesquisa Híbrida

A sofisticação necessária para a correta implementação do nosso objetivo exigiu não só a convencional pesquisa textual por palavras-chave, mas sim a Pesquisa Híbrida, que além do método anterior também recorre à pesquisa vetorial para refinar a performance do mesmo. Assim, a Pesquisa Híbrida funde a precisão léxica proveniente da pesquisa textual com a compreensão conceptual proveniente da pesquisa vetorial.

A pesquisa textual é essencial para identificar termos específicos, como nomes de marcas, modelos, entre outros, enquanto a pesquisa vetorial captura conceitos, por exemplo, para "roupa quente" associaria a casacos de inverno, de lã, entre outros.

Esta versatilidade garante que o sistema seja capaz de lidar corretamente com sinónimos, o que resolve as limitações de ambos os métodos quando utilizados individualmente.

Pesquisa Textual

O motor de pesquisa textual recorre ao algoritmo **Best Matching 25 (BM25)**, utilizado internamente pelo *Azure AI Search* como algoritmo de ranking para pesquisa *full-text*. O BM25 é uma evolução probabilística do **TF-IDF** que introduz a saturação da frequência do termo e a normalização do comprimento do documento. Originalmente, o modelo baseia-se no TF-IDF, cujo score para uma consulta Q num documento D é dado por:

$$\text{score}(D, Q) = \sum_{q_i \in Q} \text{tf}(q_i, D) \cdot \text{idf}(q_i)$$

- $\text{tf}(q_i, D)$: Frequência do termo q_i no documento D ;
- $\text{idf}(q_i)$: Importância inversa do termo no conjunto total de documentos.

Onde a componente de frequência inversa do documento é tipicamente definida como:

$$\text{idf}(q_i) = \log \left(\frac{N}{n(q_i)} \right)$$

- N : Número total de documentos no catálogo;
- $n(q_i)$: Número de documentos que contêm o termo q_i .

Em contraste, o score BM25 evolui esta lógica para:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

onde a componente IDF do BM25 utiliza uma variante suavizada no motor de busca:

$$\text{IDF}(q_i) = \log \left(\frac{N - n(q_i) + 0,5}{n(q_i) + 0,5} + 1 \right)$$

- $f(q_i, D)$: Frequência bruta do termo q_i no documento D ;
- k_1 : Parâmetro de saturação (controla o limite de influência da repetição de uma palavra);
- b : Parâmetro de normalização (ajusta o peso do comprimento do documento);
- $|D|$: Comprimento do documento atual (número de termos);
- avgdl : Comprimento médio de todos os documentos no índice de pesquisa.

Os parâmetros k_1 e b são geridos internamente pelo *Azure AI Search*, não sendo configuráveis diretamente pelo utilizador. O sistema opera sobre este ranking com **SearchMode.All**, o que impõe uma conjunção lógica (AND) entre todos os termos da consulta, privilegiando a precisão em detrimento do *recall*. No caminho *text-only*, é ainda aplicada uma expansão *fuzzy* e *wildcard* sobre cada termo (**termo***), o que permite tolerância a erros tipográficos e correspondências parciais.

Pesquisa Vetorial & Similaridade

Para a dimensão semântica, o sistema transforma as descrições e metadados dos produtos em vetores de alta dimensionalidade através do modelo de *embedding* **text-embedding-3-small** (*Azure OpenAI*). Este modelo codifica o significado semântico de sequências de texto em vetores densos, possibilitando que a proximidade entre a consulta do utilizador e o produto seja calculada através da **Similaridade de Cosseno**. Esta métrica foca-se na orientação dos vetores no espaço vetorial em vez da sua magnitude, garantindo que conceitos semanticamente relacionados sejam identificados mesmo sem sobreposição léxica direta.

Por exemplo, embora "calçado" e "tênis" não partilhem a mesma raiz léxica, são geometricamente próximos no espaço latente por serem conceitualmente semelhantes, partilhando, assim, uma elevada afinidade semântica. A fórmula da Similaridade de Cosseno é definida como:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- $\mathbf{A} \cdot \mathbf{B}$: Representa o produto escalar entre o vetor da consulta (**A**) e o vetor do produto (**B**);
- $\|\mathbf{A}\| \|\mathbf{B}\|$: Corresponde ao produto das normas (comprimentos) de cada vetor, utilizado para a normalização;
- θ : É o ângulo entre os dois vetores; quanto menor o ângulo, mais próximo de 1 está o cosseno e maior é a semelhança;
- n : Indica a dimensionalidade do espaço de *embeddings* (o número de coordenadas de cada vetor).

Para que esta comparação seja computacionalmente viável em catálogos de grande dimensão, o *Azure AI Search* utiliza o algoritmo **HNSW (Hierarchical Navigable Small World)** para busca aproximada de vizinhos mais próximos (*Approximate Nearest Neighbor*). O HNSW constrói um grafo hierárquico multicamada sobre o espaço vetorial, permitindo que as consultas naveguem das camadas mais esparsas para as mais densas, convergindo eficientemente para os vetores mais próximos sem necessitar de uma comparação exaustiva com todos os documentos indexados.

Reciprocal Rank Fusion (RRF)

Para combinar os resultados provenientes das pesquisas textual e vetorial num ranking único e coerente, o sistema utiliza o algoritmo **Reciprocal Rank Fusion (RRF)**. Este método é particularmente robusto porque não tenta normalizar as pontuações brutas de cada motor (que possuem escalas matemáticas incomparáveis), mas foca-se nas posições relativas que cada item ocupa nas diferentes listas. O RRF garante que documentos que aparecem consistentemente bem classificados em ambas as frentes de pesquisa subam para o topo da resposta final entregue ao utilizador.

A pontuação RRF para um documento d é calculada através da seguinte expressão:

$$\text{RRFscore}(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)}$$

- R : Conjunto das listas de rankings a serem fundidas (neste caso, as listas das pesquisas textual e vetorial);
- $r(d)$: Índice do documento d na lista R específica;
- k : Constante de suavização ($k = 60$ no *Azure AI Search*) utilizada para mitigar o impacto de documentos que aparecem em índices muito baixos no rank;
- D : Conjunto total de documentos recuperados em todas as listas de entrada.

Após a fusão RRF, o sistema aplica um limiar mínimo de *score* (**ProductMinScore**) para filtrar documentos cuja relevância combinada seja insuficiente, garantindo que apenas resultados com significância estatística adequada sejam devolvidos ao utilizador.

Orquestração via LLM: gpt-4.1-nano

O modelo **gpt-4.1-nano** atua como o núcleo cognitivo do sistema, sendo responsável por interpretar a intenção do utilizador e gerar respostas contextualizadas. Baseado na arquitetura Transformer — uma rede neuronal que processa sequências de dados em paralelo através de mecanismos de atenção —, o modelo utiliza o mecanismo de **Self-Attention** para avaliar a importância relativa de cada termo numa sequência, permitindo uma compreensão profunda do contexto linguístico. A escolha da variante "nano" justifica-se pela sua baixa latência, elevada eficiência na extração de entidades, transformando linguagem natural em comandos técnicos estruturados (filtros OData e vetores) em milissegundos e obviamente pelo seu custo inferior.

O mecanismo de atenção que sustenta esta orquestração é definido pela seguinte fórmula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Q (*Query*): Representa a matriz de consulta (pergunta ou o estado atual do processamento);
- K (*Key*): Representa a matriz de chaves (informação disponível contra a qual a consulta é comparada);
- V (*Value*): Representa a matriz de valores (informação que será extraída se houver correspondência entre Q e K);
- d_k : Indica a dimensão das chaves, funcionando como um fator de escala para evitar gradientes excessivamente pequenos na função *softmax*;
- softmax: Função de ativação que normaliza os pesos de atenção, garantindo que a sua soma seja igual a 1.

Processo de Reformulação de Queries

A etapa final da metodologia consiste na tradução da intenção do utilizador em consultas estruturadas. Através de *Prompt Engineering* com instruções estruturadas (*Zero-Shot*), o LLM recebe um *system prompt* detalhado que define o formato JSON esperado, as regras de validação e o vocabulário aceite para os atributos de pesquisa, sem necessidade de exemplos explícitos de *input/output*. Com base nestas instruções, o modelo decompõe frases em linguagem natural num vetor para pesquisa semântica e filtros lógicos para restrições categóricas. Este processo garante que a navegação no catálogo combine a flexibilidade conceptual da pesquisa vetorial com o rigor técnico da filtragem de metadados, assegurando uma correspondência precisa com a necessidade do utilizador.

Processo de Diálogo & Gestão Multi-turno

Estrutura do Diálogo & Gestão de Contexto

O fluxo conversacional é processado em duas fases distintas para otimizar a precisão e o consumo de recursos. A **Phase 1** atua como um filtro de intenção; interações triviais como saudações ou consultas fora do domínio são interceptadas por mensagens predefinidas. Na **Phase 2**, o sistema gere diálogos *multi-turno* utilizando o contexto anterior. O LLM condensa o histórico num `'context_summary'`, permitindo que o sistema mantenha a coerência em pedidos sucessivos — por exemplo, se o utilizador solicitar "azul" após uma pesquisa por "t-shirts", o orquestrador preserva a categoria "t-shirt" e adiciona o novo parâmetro de pesquisa.

Separação de Atributos

Um ponto crítico na arquitetura é a separação das responsabilidades de extração. O sistema isola os `'search_attributes'` da intenção principal, enquanto o par **Texto + Vetor** carrega o peso da interpretação semântica da necessidade do utilizador. Os atributos extraídos são convertidos em filtros determinísticos: filtros OData com operador `eq` para campos como `gender`, `audience` e `season`, e `search.ismatch` (*full-text match* no campo) para campos como `brand`, onde a correspondência exata é demasiado frágil face às variações do catálogo. Esta abordagem dual garante que restrições de preço ou marcas específicas sejam respeitadas com elevada fiabilidade, complementando a natureza aproximada da pesquisa puramente vetorial.

Integração Cloud & Precisão NLP

A robustez do processamento de linguagem natural é assegurada pela integração profunda na infraestrutura *Azure Cloud*. A sinergia entre o *Azure OpenAI* e o *Azure AI Search* permite que as transformações matemáticas discutidas anteriormente ocorram com latência mínima. Esta infraestrutura providencia as ferramentas necessárias para que a sensibilidade linguística ao Português de Portugal seja aplicada tanto na fase de análise da *query* como na recuperação de documentos, garantindo que as nuances locais do mercado sejam preservadas.

Conclusão

Arquitetura, Escalabilidade & Fundamentos

O sucesso deste projeto está diretamente associado às escolhas arquiteturais efetuadas. A refatoração em camadas (**Api, Application e Custom**) conferiu ao sistema uma modularidade que separa a lógica de negócio da infraestrutura de dados. Esta organização, aliada a uma preparação rigorosa e aos fundamentos matemáticos da Pesquisa Híbrida, resulta numa solução altamente escalável. O sistema não depende de correspondências exatas de palavras-chave, mas sim de uma infraestrutura capaz de crescer em volume de dados sem perder a precisão na recuperação de informação.

Validação de Conceito & Garantia de Qualidade

A capacidade de gerir contextos complexos, extrair parâmetros técnicos de linguagem natural e executar pesquisas semânticas com sucesso prova que o modelo híbrido é superior às abordagens tradicionais de e-commerce. A garantia de qualidade reside no equilíbrio entre a criatividade do LLM e o rigor dos filtros lógicos sobre o índice de pesquisa. Em suma, o protótipo demonstra ser uma ferramenta robusta, preparada para transformar a experiência de navegação estática numa interação dinâmica e inteligente.